Import Libraries

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
```

Load and Preprocess Data

```
df=pd.read_csv('/content/IMDB Dataset.csv', engine='python')
```

```
import re

def clean_text(text):
    # Remove HTML tags
    text = re.sub(r'<.*?>', '', text)
    # Remove punctuation and numbers, keep only alphabetic characters and spaces
    text = re.sub(r'[^a-zA-Z ]', '', text)
    # Convert to lowercase
    text = text.lower()
    return text

print("clean_text function defined successfully.")
```

```
clean_text function defined successfully.
```

```
df['review'] = df['review'].apply(clean_text)
df.head()
```

	review	sentiment
0	one of the other reviewers has mentioned that ...	positive
1	a wonderful little production the filming tech...	positive
2	i thought this was a wonderful way to spend ti...	positive
3	basically theres a family where a little boy j...	negative
4	petter matteis love in the time of money is a ...	positive

Next steps: Generate code with df New interactive sheet

```
import nltk
from nltk.corpus import stopwords

# Download the 'stopwords' corpus if not already downloaded
try:
    stopwords.words('english')
except LookupError:
    nltk.download('stopwords')
    print("NLTK 'stopwords' corpus downloaded.")
else:
    print("NLTK 'stopwords' corpus already available.")

print("nltk and stopwords imported successfully.")
```

```
NLTK 'stopwords' corpus already available.
nltk and stopwords imported successfully.
```

```
from nltk.corpus import stopwords

def remove_stopwords(text):
    stop_words = set(stopwords.words('english'))
    words = text.split()
    filtered_words = [word for word in words if word not in stop_words]
    return ' '.join(filtered_words)
```

```
# Apply the function to the 'review' column
df['review'] = df['review'].apply(remove_stopwords)
df.head()
```

	review	sentiment
0	one reviewers mentioned watching oz episode yo...	positive
1	wonderful little production filming technique ...	positive
2	thought wonderful way spend time hot summer we...	positive
3	basically theres family little boy jake thinks...	negative
4	petter matteis love time money visually stunni...	positive

Next steps: Generate code with df New interactive sheet

```
df.head()
```

	review	sentiment
0	one reviewers mentioned watching oz episode yo...	positive
1	wonderful little production filming technique ...	positive
2	thought wonderful way spend time hot summer we...	positive
3	basically theres family little boy jake thinks...	negative
4	petter matteis love time money visually stunni...	positive

Next steps: Generate code with df New interactive sheet

Feature Extraction

```
tfidf_vectorizer = TfidfVectorizer(max_features=5000) # You can adjust max_features as needed
```

```
X = tfidf_vectorizer.fit_transform(df['review'])
Y=df['sentiment'].apply(lambda x: 1 if x == 'positive' else 0)
vocabulary_size = len(tfidf_vectorizer.vocabulary_)
print(X.shape)
print(Y.shape)
print(f"Vocabulary size: {vocabulary_size}")
```

```
(50000, 5000)
(50000,)
Vocabulary size: 5000
```

Train Logistic Regression Model

```
X_train, X_test, y_train, y_test = train_test_split(X, df['sentiment'], test_size=0.2, random_state=42)

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}")
```

```
Shape of X_train: (40000, 5000)
Shape of X_test: (10000, 5000)
Shape of y_train: (40000,)
Shape of y_test: (10000,)
```

```
model = LogisticRegression(max_iter=1000) # Increased max_iter for convergence
model.fit(X_train, y_train)

print("Logistic Regression model trained successfully.")
```

```
Logistic Regression model trained successfully.
```

Model Evaluation

```
y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy:.4f}")

print("\nClassification Report:")
print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

```
Accuracy: 0.8873

Classification Report:
              precision    recall  f1-score   support

    negative       0.90      0.87      0.88      4961
    positive       0.88      0.90      0.89      5039

    accuracy                           0.89     10000
   macro avg       0.89      0.89      0.89     10000
weighted avg       0.89      0.89      0.89     10000


Confusion Matrix:
[[4328  633]
 [ 494 4545]]
```